Reflection of the paper:
Competitive Non-Clairvoyant KV-Cache Scheduling for LLM Inference

Ka Yiu Hung

September 2026

Abstract

This is the reflection of the paper after a deeper review that was mentioned during the previous meeting. Link of the Paper is in here.

Contents

1 Reflections and Insights **2**

1.1 General . . . 2

1.2 Knowledge needed before studying the proofs of this paper . . . 3

1.2.1 i$\tau \equiv \ell \pmod{k} \iff \frac{i\tau}{gcd(\tau,k)} \equiv \frac{\ell}{gcd(\tau,k)} \pmod{\frac{k}{gcd(\tau,k)}}$ . . . 3

1.2.2 Equivalence Class . . . 3

1.2.3 τ-$\lceil \frac{\tau}{2} \rceil = \lfloor \frac{\tau}{2} \rfloor$ . . . 4

1.2.4 $\left\lceil \frac{N_{>p}\tau_p}{k_p} \right\rceil \leq N_{>p} \left\lceil \frac{\tau_p}{k_p} \right\rceil$ . . . 4

1.2.5 $\left\lceil \frac{N_{>p}\gamma_p}{k_p} \right\rceil \leq N_{>p}\gamma_p + 1$ and $n_p \frac{\tau_p}{k_p} < \left\lfloor \frac{n_p\tau_p}{k_p} \right\rfloor + 1$ . . . 4

1.2.6 $n \left\lceil \frac{n}{k} \right\rceil \leq 2 \sum_{u=1}^{n} \left\lceil \frac{u}{k} \right\rceil$ . . . 4

1.2.7 $\lfloor a+b \rfloor \leq \lfloor a \rfloor + \lceil b \rceil$, $\lceil a+b \rceil \leq \lceil a \rceil + \lceil b \rceil$ and $\lceil \sum a \rceil \leq \sum \lceil a \rceil$ . . . 4

1.2.8 $\left\lceil \frac{x}{y} \right\rceil \leq \left\lfloor \frac{x-1}{y} \right\rfloor + 1$ . . . 4

1.2.9 $\tau_t \left\lfloor \frac{i}{k_t} \right\rfloor \leq \left\lfloor \frac{i\tau_t}{k_t} \right\rfloor$ . . . 4

1.2.10 $\lfloor \frac{x}{2} \rfloor \geq \frac{x-1}{2}$ . . . 5

2 Problems unresolved **5**

2.1 Overview . . . 5

2.2 Proof of Proposition EC. 1 . . . 5

3 Resolved Queries and Inspirations **7**
3.1 Overview . . . 7
3.2 Resolved Queries . . . 8
3.3 Consolidated Knowledge and Theoretical Takeaways . . . 8
3.4 Other Inspirations . . . 13

4 Physical Meaning and Exploration of how those Theorems come up **14**
4.1 Overview . . . 15
4.2 Lists of Theorems and some potential Interpretation . . . 15
4.2.1 Proof of Theorem 1 from Lemma 1 to 4 . . . 15
4.2.2 Proof of Theorem 3 from Lemma 5 to 7 . . . 16
4.2.3 Proof of Theorem 2 from Lemma 8 to 11 . . . 18
4.2.4 EC. 2.1 for Theorem 1 . . . 20
4.2.5 EC. 2.2 for Theorem 1 . . . 20

5 Conclusion and Acknowledgments **21**

6 References **21**

Outline

1 Reflections and Insights

Back to Top

1.1 General

This paper explicitly states that preemption (kill and restart) could be a better option instead of redemption, which is clearly mentioned in P. 8 and P. 13 instead of swapping to the slower memory or keep it till available memory capacity. It is a little bit counter-intuitive. It further mention about the sawtooth shape of the memory usage and further explain the need of staggered scheduling by allowing jobs begin and end in between of other jobs processing or waiting.

I feel inspired on how those upper bound could be magnificent aligned perfectly including those $n_p S_p$, Q_p and how to merge those terms in terms of OPT in this paper by showing the bound of GSA is some coefficient times the GBA upper bound and then show it is having the competitive ratio to OPT.

It also allows me to learn a lot by quoting a few previous studies of this field and did provide some potential research by adding more settings. Though I prefer studying those Math parts instead of those wordy parts, those are actually user for a beginner to study about.

In conclusion part, the paper provided some possible future direction. Including heterogeneous prompt lengths, online arrivals. Approaching to more realistic scenarios step by step by adding more conditions.

1.2 Knowledge needed before studying the proofs of this paper

Though it could still be able to understand most or even all of the proofs in this paper. It would be better to understand or revise some of those knowledge for preparing studying. They are mainly in number theory and could also be related to abstract algebra. Though it might not be necessary to study this part if only looking for the flow of the paper or looking at the performance of the algorithm.

1.2.1 $\mathbf{i\tau \equiv \ell \pmod{k}} \iff \frac{i\tau}{gcd(\tau,k)} \equiv \frac{\ell}{gcd(\tau,k)} \pmod{\frac{k}{gcd(\tau,k)}}$

It is used in Lemma 1 in P. 16.

1.2.2 Equivalence Class

It is used in Lemma 1 in P. 16.

|Equivalence class|=$\frac{k}{gcd(\tau,k)}$

1.2.3 τ-$\lceil \frac{\tau}{2} \rceil = \lfloor \frac{\tau}{2} \rfloor$

It is used in Theorem 1 Proof EC. 2.1 in P. 50.

1.2.4 $\left\lceil \frac{N_{>p}\tau_p}{k_p} \right\rceil \leq N_{>p} \left\lceil \frac{\tau_p}{k_p} \right\rceil$

It is used in Theorem 2 Proof of Lemma 8 in P. 34. Notice that the theorem holds when $N_{>p}, k_p \in \mathbb{N} \cup \{0\}, N_{>p} \geq 0, k_p \geq 1, \tau_p \in \mathbb{R}, \tau_p \geq 1$. Though those conditions could be relaxed as $N_{>p} \in \mathbb{N} \cup \{0\}, N_{>p} \geq 0, k_p, \tau_p \in \mathbb{R}, k_p > 0$.

Similar technique also used in Theorem 2 Proof of Lemma 10 in P. 36.

1.2.5 $\left\lceil \frac{N_{>p}\gamma_p}{k_p} \right\rceil \leq N_{>p}\gamma_p + 1$ **and** $n_p \frac{\tau_p}{k_p} < \left\lfloor \frac{n_p \tau_p}{k_p} \right\rfloor + 1$

Obvious. It is used in Theorem 2 Proof of Lemma 9 in P. 34.

1.2.6 $n \left\lceil \frac{n}{k} \right\rceil \leq 2 \sum_{u=1}^{n} \left\lceil \frac{u}{k} \right\rceil$

Though it is directly from the proof of Lemma 11 in P. 35. It could be a good tool.

1.2.7 $\lfloor a+b \rfloor \leq \lfloor a \rfloor + \lceil b \rceil$, $\lceil a+b \rceil \leq \lceil a \rceil + \lceil b \rceil$ **and** $\lceil \sum a \rceil \leq \sum \lceil a \rceil$

It is used in Theorem 2 Proof of Lemma 10 in P. 36.

1.2.8 $\left\lceil \frac{x}{y} \right\rceil \leq \left\lfloor \frac{x-1}{y} \right\rfloor + 1$

It is used in Theorem 2 Proof of Lemma 10 in P. 36.

1.2.9 $\tau_t \left\lfloor \frac{i}{k_t} \right\rfloor \leq \left\lfloor \frac{i\tau_t}{k_t} \right\rfloor$

It is used more than once in Theorem 2 Proof of Lemma 10 in P. 37.

1.2.10 $\lfloor \frac{x}{2} \rfloor \geq \frac{x-1}{2}$

Obvious. It is used in Theorem 1 Proof of E.C. 2.1 in P. 50.

2 Problems unresolved

Back to Top

2.1 Overview

While I believe I have grasped the foundational proofs and structural arguments established in the main text, there is a intricate mathematical transitions in the Appendix remain challenging to fully reconcile.

2.2 Proof of Proposition EC. 1

In Proposition EC. 1 (P. 52∼53). There is a part that make me believe that there is an alternative path for proving.

Recall PROPOSITION EC.1.

PROPOSITION EC.1 (Asymptotic lower bound of 2 for SIMS). There exists a family of identical-job instances such that the approximation ratio of SIMS is at least 2-ϵ even in the asymptotic regime where M $\to +\infty$, n $\to +\infty$, and M = o(n).

In the second step, which showing that

$$k^*(\tau, s) \geq \left\lfloor \frac{2M - o + 1}{o + 1} \right\rfloor = \left\lfloor 2B - \frac{2B - 1}{o + 1} \right\rfloor$$

However, I was unable to get the RHS of this inequality. If I follow the steps, it would be

$$k^*(\tau, s) \geq \left\lfloor \frac{2M - o + 1}{o + 1} \right\rfloor$$

$$= \left\lfloor \frac{2B \cdot o - o + 1}{o+1} \right\rfloor$$
$$= \left\lfloor \frac{2B \cdot (o+1) - 2B - o + 1}{o+1} \right\rfloor$$
$$= \left\lfloor 2B - \frac{2B + o - 1}{o+1} \right\rfloor$$

, which should be smaller than

$$\left\lfloor 2B - \frac{2B - 1}{o+1} \right\rfloor$$

instead, provided that o≥1, which could not draw the conclusion of that line of inequality.

Still, I did my own proof for that part.

$$k^*(\tau, s) = \left\lfloor 2B - \frac{2B + o - 1}{o+1} \right\rfloor$$
$$= \lfloor 2B \rfloor + \left\lfloor -\frac{2B + o - 1}{o+1} \right\rfloor$$

Notice that they could be separated by the first term is the integer, which the floor function actually only affecting the second term. Then continue from that step. Also, notice that the negative sign need to be put inside the floor function or else the floor function would decrease the total value would be increased instead. Though we could also put the negative sign outside by using the ceiling function on the second term. After that we have

$$= 2B + \left\lfloor -\frac{2B + o - 1}{o+1} \right\rfloor$$
$$\geq 2B - \frac{2B + o - 1}{o+1}$$
$$\implies \frac{k}{B} \geq \frac{2B - \frac{2B+o-1}{o+1}}{B}$$

$$= 2 - \frac{2 + \frac{o}{B} - \frac{1}{B}}{o+1}$$

$$\implies \frac{k}{B} \cdot (1-\epsilon) \geq \left(2 - \frac{2 + \frac{o}{B} - \frac{1}{B}}{o+1}\right)(1-\epsilon)$$

$$= 2 - 2\epsilon - \frac{2 + \frac{o}{B} - \frac{1}{B}}{o+1} + \left(\frac{2 + \frac{o}{B} - \frac{1}{B}}{o+1}\right)\epsilon$$

$$= 2 - 2\epsilon - \left(\frac{2}{o+1} + \frac{\frac{o}{B}}{o+1} - \frac{\frac{1}{B}}{o+1}\right) + \left(\frac{2}{o+1} + \frac{\frac{o}{B}}{o+1} - \frac{\frac{1}{B}}{o+1}\right)\epsilon$$

$$= 2 - \left(\frac{2}{o+1} + \frac{\frac{o}{B}}{o+1} - \frac{\frac{1}{B}}{o+1}\right) + \left(-2 + \frac{2}{o+1} + \frac{\frac{o}{B}}{o+1} - \frac{\frac{1}{B}}{o+1}\right)\epsilon$$

$$= 2 - \left(\frac{2}{o+1} + \frac{\frac{1}{B}}{1+\frac{1}{o}} - \frac{\frac{1}{B}}{o+1}\right) + \left(-2 + \frac{2}{o+1} + \frac{\frac{1}{B}}{1+\frac{1}{o}} - \frac{\frac{1}{B}}{o+1}\right)\epsilon$$

, which the first line of the proof in the paper already assumed $o \to +\infty$ and $B \to +\infty$, then

$$\implies = 2 - \left(\frac{2}{o+1}\right) + \left(-2 + \frac{2}{o+1}\right)\epsilon \quad \text{when } B \to +\infty$$

$$= 2 - 2\epsilon \quad \text{when } o \to +\infty$$

$$= 2 - \epsilon \quad \text{when } \epsilon \text{ is sufficiently small.}$$

3 Resolved Queries and Inspirations

Back to Top

3.1 Overview

This section outlines the conceptually enriching queries that arose during my deep review. While navigating the intricate formulations presented several structural challenges, these have been successfully resolved through careful deduction. To maintain conciseness, this section focuses strictly on the insights and extensions that are more inspirational and

tactical derived from the text.

3.2 Resolved Queries

Q1: If there is Paged Attention, why still need to care about the external fragmentation?

A1: Though Paged Attention solve the tokens level of external fragmentation, but still causing the metadata external fragmentation.

3.3 Consolidated Knowledge and Theoretical Takeaways

1. Approximation Ratio is for offline algorithm, competitive ratio is for online algorithm.

2. $\mathcal{O}(1)$-competitive algorithm could be referring to a algorithm having constant competitive ratio (constant bounded) regardless of the amount of jobs.

3. Attention QKV without using KV-cache are having $\mathcal{O}(n^2)$ in each step of computation, summing up with $\mathcal{O}(n^3)$ in total decoding process of the whole request.

4. Thus, KV-cache is having $\mathcal{O}(n)$ in each step of computation, summing up with $\mathcal{O}(n^2)$ in total decoding process of the whole request. Pros: No need to recompute. Cons: Storing more K and V as decode size growing.

5. Revised some contents about multi-head attention, though they have been taught in Computer Vision and Statistical Machine Learning courses. It would generate more Q, K, V to show the correlations of each data point. Q heads: K heads: V heads in multi-head attention is 1:1:1 same as single head. There are some more others different attention variations with different mechanisms. Multi-head Latent Attention.

   1. Multi-query attention, which the ratio of Q : K : V= N : 1 : 1.
   2. Grouped-query attention, which is grouping a few queries together, for example 8 queries in a group, then the ratio of Q : K : V= N : G : G (G is

the group numbers and G<N).

Notice in those model, normally nowadays the hidden layers in the transformers should having the same value of d. Though some special transformer including tunnel-transformer does not follow that. The disadvantages are unable to easily express it in the 4-dim tensors in form of [layers, number of words, number of heads, number of d].

6. PagedAttention: Reference on virtual memory paging in the operating system.

7. In past LLM, even including HuggingFace, they would normally schedule a largest possible size of memory to each user by assuming the users would use all of it. It would cause memory space wasted easily. Solutions include: 1. Logical KV-blocks by grouping tokens in a block. 2. Physical KV-Blocks: Built blocks in physical GPU and allow disputatiously placed in the GPU. 3. Block Table: Record the physical address of each logical blocks. Address could be shared if different requests are using the same block of tokens.

8. Staggered pipeline.

9. Concurrency.

10. Batching strategy, though it also used in the previous paper studied, it just raised the awareness personally. Also need to balance about the number of batches and the requests in each batch.

11. Though GPU and VRAM have high transmission speed. GPU to PCLE speed is extremely slow and could cause lagging. This could be the reason why choosing kill and restart rather than suspension temporarily. High Bandwidth Memory (HBM). NVLink. Tensor cores. CPU search for the address and ask GPU to capture those blocks or tell GPU which location still have space.

12. Large-Memory (Regimes). $M, n \to +\infty$, M=$o(n)$. Large memory instances $M \to +\infty$ in this paper. Worst case $k_{min} = 1$, large memory regime $k_{min} \to +\infty$

13. Speculative Decoding, larger model generation with the smaller model.

14. Raise the awareness to reduce the peak of the memory usage, which here mainly referring to make the peak value smaller, not the time that having peak value smaller.

15. Sawtooth usage curve.

16. Tradeoff of the value of the scaling factor α. Smaller α: Fine-grained with too many classes, with only few requests in each class. Concurrency is reduced. Increase the frequency of kill and restart. Larger α, coarser, larger time slice, easily to have more time space that the system is being idle. Thus increase the total latency.

17. Raise the area concept to view the relationship of memory and the time usage.

18. Approximation class (APX): If a NP-hard problem could came up with polynomial time algorithm with constant approximation ratio (offline algorithm), then it is in APX.

19. Polynomial-Time Approximation Scheme (PTAS): Approximation ratio infinitely tends to 1.

20. APX-hard: Those belong to APX problem that could never have approximation ratio tends to 1.

21. Recall the 2D Packing problem. However, there are variants including strip packing instead of bin packing which already included in the reflection of the previous paper. Strip packing is also proved as approximation hardness by this paper.

22. Round-robin for flow time objectives.

23. Multi-level-feedback-style, or Multilevel Feedback Queue (MLFQ).

24. Multi-level-feedback-style Adversarial model is mainly operated by those smaller model for filtering and then reverify by larger model. Defended by weaker model then stronger model. Combine the concept of MLFQ and Adversarial model.

25. Assumption of costless preemption.

26. In LLM, it could be solving 2D triangular packing+non-clairvoyant+lost all progress by kill and restart unlinke 2D rectangle packing.

27. Robustness-consistency-smoothness tradeoffs. Robustness: Worst case CR or approx. ratio. Consistency: The CR or approx. ratio when the pred of the decode size is 100% correct. Smoothness: Sensitivity of pred. error on the decode size.

28. Adversarial could means error intentionally made. Stochastic means irreducible noise.

29. Throughput.

30. Fluid Approximation.

31. Prompt Caching.

32. Tail latency: The slowest part latency (e.g. P95).

33. Prefix Caching.

34. Min-sum Bin Packing.

35. $\frac{M}{A(\tau,s)}$ can be interpreted as an upper bound on the throughput (jobs per round) achievable by any feasible schedule—particularly the optimal clairvoyant scheduler. $\frac{k^*(\tau,s)}{\tau}$ represents the effective throughput of SPS.

36. If we assume something is the k-th to happen, we may notice about the distribution of happening in k-th and may find the expectation value by k after having the formula with input k. This is inspired by P. 22 of the paper, which is the example 2 by assuming the longer job is the k-th job and notice that k is uniformly distributed then could find the expectation of the formula by the distribution on k.

37. The areas spanned by the first n jobs $\leq$ M*C_n, the n-th job completion time.

38. Separating the floor function and ceiling function in summation by those who return the original value which the original value is integer and those does not return the original value which originally were not integers and process them.

39. Kernel Thread Contention, occurs when multiple kernel-level threads compete simultaneously for a limited system resource, forcing one or more threads to wait and slowing down overall system execution. occurs when multiple kernel-level threads compete simultaneously for a limited system resource. Unlike user-space thread contention (which happens within an isolated application), kernel thread contention impacts the operating system core, directly affecting CPU scheduling, hardware interrupts, and system-wide performance.

40. Locking overhead, the extra CPU time, memory, and system resources used to request, maintain, and release. It could also from the concurrency bottleneck when processing in both GPU and CPU and also the data transferring.

41. Global VRAM. The one with largest capacity but also produce the largest latency. If multiple jobs are requesting for memory space or releasing space (cudaMalloc/ cudaFree), then mutex lock may triggered and causing relatively high latency.

42. SRAM.

43. Mutex lock, is used on the host side (CPU) or in a multi-threaded environment to control access to GPU resources, when one thread locks the GPU resources, other threads cannot submit tasks to the GPU. This CPU-side waiting causes the GPU to enter a brief state of "starvation," rendering it unable to process tasks continuously and thereby driving up overall latency.

44. Tensor Parallelism (Ultra low latency, all-reduce/ all-gather is needed, mainly for locally by having frequent communication) vs Pipeline Parallelism (Cross-machine processing, need to wait for the previous GPU processed, could not reduce single request). Then, scheduling (including 1F1B) is needed to reduce the computation bubbles for pipeline Parallelism in those modern engine (e.g. Megatron-LM), it could still be ex-

plored. One of the Solution for Modern LLM Inference is 3D Parallelism (TP + PP + DP). In modern actual production environments (e.g. when running ultra-large models over 405B), engineers combine them using hybrid parallelism. Within a single machine. They use TP (e.g. 4 or 8 GPUs) to leverage NVLink, achieving high-speed intra-node matrix parallelism with zero bubbles. Across multiple machines. They use PP to daisy-chain the machines together, bypassing the bottleneck of insufficient cross-node network bandwidth. If chasing low TTFT, prioritize optimizing your TP strategy. If chasing long context or high batch throughput, use PP or Context Parallelism.

45. Discuss the purposes in both systems and algorithmic perspective. It is inspired from P. 48, ec5, Trace-Driven Power-of-Two Rounding Results.

46. Chunked-Prefill Slot Allocation Lock.

47. Inspired that could use 2^n to align the decode size.

48. External fragmentation: Memory space fragmented and could not be used even if in total memory capacity is sufficient.

49. Mean flow time.

50. Scalability and effectiveness.

51. The peak memory usage occurs when the earliest active job is about to finish in staggered pipeline.

3.4 Other Inspirations

1. I have noticed that from this paper and the paper studied before (LLM Serving Optimization with Variable Prefill and Decode Lengths), they are mainly discussing about the total latency in all requests. However, I believe that in the users perspective, the tolerance of the waiting time could also be depends on their expectation based on their

prefill size, which is the input length and also their expected decoding size. Though it may not be necessary to evaluate them in the mathematical part, it may be useful to think of some variations of the suggested algorithm that could make it easier to work in industry.

2. Could there any policies that by handling those requests that are being killed and restart multiple of times even in non-clairvoyant cases by assuming it could not be processed in those smaller phases and put it in behind the queue?

3. As from the paper proposed, could also be studied the LLM system runs in multiple-machine. That make me think of queuing theory in multiple systems.

4. Some intermediate settings could be discussed. Including try to allow online arrival but by constraining the limited arrivals, or size of requests in each small time slice window, which allow to set τ_p dynamically. Speculative decoding, which could have different shape of memory usage by time than only sawtooth shape by having memory usage decrease even for a single request. Localized homogeneous Multi-GPU Routing by only assuming those GPU are in the same memory space, which have some more variations on when should the requests could be transferred.

4 Physical Meaning and Exploration of how those Theorems come up

Back to Top

4.1 Overview

This section outlines some potential physical meaning behind those theorems, though it could be Section 3. However, for this paper, they are relatively implicit and prefer placing them in a separated section for exploration.

4.2 Lists of Theorems and some potential Interpretation

4.2.1 Proof of Theorem 1 from Lemma 1 to 4

Flowtime of SPS by taking out the floor function. OPT flowtime lower bounded by summing all jobs $\frac{Area(\tau,s)}{M}$ ($A(\tau, s)$ is an obvious minimum area needed in autoregressive mechanism), then 2 terms by the upper bound of SPS and recalling definition 1 of Area by o and s to get the upper bound of $\frac{SPS}{OPT}$.

In the paper, it mentioned that $\frac{M}{A(\tau,s)}$ can be interpreted as an upper bound on the throughput (jobs per round) achievable by any feasible schedule—particularly the optimal clairvoyant scheduler. Meanwhile, $\frac{k^*(\tau,s)}{\tau}$ represents the effective throughput (How many jobs could be processed in parallel/Time Rounds) of SPS. In this sense, Lemma 3 lower bound the optimal total flow time via its throughput limit, while Lemma 4 shows that SPS achieves nearly optimal throughput, up to a small multiplicative factor in the parallelism dimension.

Lemma 4 using the inverse way by showing left hand side is $\frac{\tau}{k^*(\tau,s)}$ instead of $\frac{k^*(\tau,s)}{\tau}$, though it normally care about the term $\frac{A(\tau,s)}{M}$, which help to derive a better bound for Lemma 3, it is to cancel out the term of M, and to compare both SPS and OPT much easier.

In Theorem 1, the upper bound should be with the terms α and k_{min}, and lemma 3 to 4 are used in looking for the lower bound of OPT.

4.2.2 Proof of Theorem 3 from Lemma 5 to 7

Recalling the steps for those Lemmas.

Lemma 5: Lower Bound for OPT, and OPT≥Within Class+Between Class. It is lower bounded by using the lowest possible value for the decode size in each job and use that to be the lower bound for the area.

Lemma 6: Similar to Lemma 4 but multi-class by different τ_p, would be discussed the differences of Lemma 4. Notice that the term α in here which is extra from Lemma 4 by multi-class is from the τ_p. whcih derive as $\tau_p = \lfloor \hat{\tau_p} \rfloor \leq \hat{\tau_p} = \beta\alpha^p$, and since we let y=$\beta\alpha^{p-1}$, and we have $\tau_p \leq \alpha y$.

Lemma 7: Upper Bound of GBA. Assuming every jobs placed in the batch is having largest possible decode size, which is τ_p, this would be the upper bound for GBA by bounding by summing for each job, the prior batch time+ the beginning processing time in the own batch in SPS. It could be summing by each batch in the summation.

Theorem 3: Approximation Ratio of GBA. Since need to compare with OPT, the upper bound of Lemma 7 for GBA need to be enlarged. It is very difficult part by noticing it could not be directly make the Lemma 7 upper bound be express larger in terms of OPT. However, noticing that the lower bound of OPT could be separated in WITHINCLASS and BETWEENCLASS, which is useful when enlarging the bound of GBA separating for Q_p and S_p. We want to make the summation be disappeared in both Q_p and S_p.

For Q_p, look at the beginning time of each job first, it could be enlarged by taking out the floor function and using Lemma 6 that enlarge the upper bound from the inverse of effective throughput $\frac{\tau_p}{k_p}$ to the $\frac{A_p}{M}$, which is the inverse of the upper bound on the throughput (jobs per round) achievable by any feasible schedule, or the lower bound of

the completion time, which is from $\frac{A_{p_j}}{M} \leq \frac{A_j}{M} \leq C_j$ by $A_j \leq MC_j$, which make the term $\frac{A_p}{M}\frac{n_p(n_p-1)}{2}$ and it $\leq \frac{A_p}{M}\frac{n_p(n_p+1)}{2}$, which make it exactly equal to the WITHINCLASS in the lower bound of OPT in Lemma 5.

Then look for $n_p\tau_p$ in Q_p, in this part, the author use a more uninhibited upper bound for aligning the OPT. In here the original term is $\sum_p n_p\tau_p$, which is only counting for the highest possible decode size for each batch. However, the upper bound in enlarge to counting each decode length of each job, and to upper bound the original τ_p by the decode size of each job in that batch times one more scaling factor α. Then by the sum o_i of those jobs, which is the least loweb bound possible for the latency of OPT because no matter of which algorithm, each job need to spend at least o_i time unit for processing the whole job. then we have $\alpha \cdot OPT$.

For n_pS_p, it is also separated in 2 terms. First look at (I), it make $N_{>q}$ to be n_p to make it easier to be controlled. However, we then have to sum all the τ_q, which q is smaller than p. Still it is bounded by p and we could upper bound it by taking out the floor function and becoming $\hat{\tau_q}$, and compute it by $\beta\alpha^n$ and become a geometric sequence, then the trick is using $\hat{\tau_p} \leq \tau_p + 1$ and $\beta \geq 1$ to cancel out the β term clearly. It then becoming a similar term as $n_p\tau_p$ in Q_p, which making an upper bound with the term OPT.

Then look at (II). Notice there is also a term with $\frac{\tau_p}{k_p}$, which could use Lemma 6 again, which is very similar to the beginning time of each job in Q_p. The enlarged terms are then also similar. Just one with WITHINCLASS and one with BETWEENCLASS. Here also show the authors did have aligned struturally decomposition from OPT and GBA.

Then summing all the terms here by the upper bound and also using the Lemma 5 to merge the WITHINCLASS and BETWEENCLASS.

4.2.3 Proof of Theorem 2 from Lemma 8 to 11

Step 1. The spillover Δ_p and phase-prefix cost T_p are defined.

Lemma 8: Upper Bound of GSA. Sharing the same Q_p in GBA. However, not using the same S_p by there is spillover term and the delay of the jobs afterwards in the same batch by those jobs that not belong to that batch. Then the upper bound of the spillover Δ_p and phase-prefix cost T_p would be discussed in Lemma 9 and Lemma 10.

Lemma 9: Upper Bound of Spillover cost Δ_p. It first show that it is upper bound by $N_{>p}n_p\frac{\tau_p}{k_p}$ and n_p and enlarge the bound for those 2 terms separately. For $N_{>p}n_p\frac{\tau_p}{k_p}$, $n_p\frac{\tau_p}{k_p} \leq \lfloor n_p\frac{\tau_p}{k_p}\rfloor+1 \leq \lfloor n_p\frac{\tau_p}{k_p}\rfloor+\tau_p$ and also notice that S_p in GBA is equal to $\sum_{q<p}(\lfloor\frac{n_q\tau_q}{k_q}\rfloor+\tau_q)$, it could make it enlarge to that term and convert it to be in terms of S_p.

Lemma 10: Upper Bound of Phase-prefix cost T_p. This requires the theorem from 1.2.6. which is the Lemma 11. By recalling the definition of T_p in Step 1, noticing that $N_{\geq q}$ could be separated into $n_q + N_{>q}$ and also using the theorem from 1.2.7 to separate it from the floor function. Then let the n_q one be (I) and $N_{>q}$ one be (II). (I) is actually equal to n_pS_p, noticing that Lemma 9 upper bound is also in terms of S_p.

Then handle (II) with $N_{>q}$, using 1.2.9 to take out the τ_p first. Then we have $N_{>q}\lceil N_{>q}/k_q\rceil$, which could use the Lemma 11, the $N_{>q}\lceil N_{>q}/k_q\rceil \leq 2\sum_{u=1}^{N>q}\lceil u/k_q\rceil$ could be interpreted as $\lceil N_{>q}/k_q\rceil$ assuming all jobs after q are processing in the degree of parallelism in phase q and assume those jobs are being processed at the same time from the beginning, then it is lower bound of the time units needed by taking out the actual τ for each job after q, or assuming all $\tau = 1$. Also, the actual time units needed should usually even be larger by the degree of parallelism should be decreasing after state q in the same memory capacity when the decode size should be increasing. Then it is the total latency of all jobs after stage q by adding the factor $N_{>q}$ to $\lceil N_{>q}/k_q\rceil$. The

$\sum_{u=1}^{N_{>q}} \lceil u/k_q \rceil = \lceil 1/k_q \rceil + \lceil 2/k_q \rceil + \lceil 3/k_q \rceil + ... + \lceil N_{>q}/k_q \rceil$ could be interpreted as first we have 1 job processing, then adding 1 job after 1 time unit by $\lfloor i\tau/k \rfloor$ by SPS, which is $\leq \lceil i\tau/k \rceil$. It then tell that the total latency needed for processing all jobs at the same time is at most 2 times the total latency from the mechanism of SPS.

After Lemma 11, separating the term $\sum_{u=1}^{N_{>q}} \lceil u/k_q \rceil$. Assuming for each u is from the r-th job in phase t. Then counting after phase q+1 from $N_{>q}$, which let $u = r + \sum_{v=q+1}^{t-1} n_v$ and separating them by $\lceil a+b \rceil \leq \lceil a \rceil + \lceil b \rceil$ by u is in the ceiling function. Then for the phase after q the $k_t \leq k_q$, which then $1/k_t \geq 1/k_q$. Also, then $1/k_v \geq 1/k_q$ by $v \in [q+1, t-1]$, then $v \geq q$. Then having $\lceil r/k_t \rceil$, which is (IIA), and $\lceil (\sum_{v=q+1}^{t-1} n_v)/k_v \rceil$, which is (IIB).

Now look at (IIA), it is using the upper bound by $\hat{\tau_p} = \beta\alpha^p$ and $\hat{\tau_p} \leq \tau_p + 1$ to have the geometric sequence same as proving $n_p S_p$ in Theorem 3.It then take out the n_t terms in the summation and add the 0 terms, which the terms here is 0, which does no change on the value, if changing r to i by dummy variable and enlarging it by placing the τ_t also in the floor function, then it is exactly Q_t. t here is dummy variable and could be changed to p to align the other terms.

Now look at (IIB), it is using the upper bound by $\hat{\tau_p} = \beta\alpha^p$ and $\hat{\tau_p} \leq \tau_p + 1$ to have the geometric sequence same as proving $n_p S_p$ in Theorem 3. It then using 1.2.8. Then we have $\sum_{q<p} \lfloor \frac{n_q \tau_q}{k_q} + \tau_q \rfloor$, which is exactly same as S_p.

Then from the upper bound of $n_p T_p$, we have both (I) and (IIB) are in terms of $n_p S_p$ and (IIA) is in term of Q_p.

Recalling that by Lemma 9, we have $n_p \Delta_p$ bounded by $n_p S_p$ and Q_p, then the GSA$\leq$ $\sum_p n_p(T_p + \Delta_p) + \sum_p Q_p$ is bounded fully in terms of $n_p S_p$ and Q_p. Also, they are having both same coefficient of $2+\frac{2}{\alpha+1}$. In detail, we have the coefficient in $n_p S_p$ and Q_p for the

upper bound of each term. 1. $n_p\Delta_p$: (1, 1). 2. Q_p: (0, 1). 3. n_pT_p: $(1+\frac{2}{\alpha-1}, \frac{2}{\alpha-1})$, which summing up are the same and could be factored, which by the terms in only n_pS_p and Q_p, which then could compare to GBA easily by the upper bound of GBA. Then it could be compare to OPT by GBA upper bound could be enlarged to in terms of OPT in Theorem 3.

4.2.4 EC. 2.1 for Theorem 1

It shows the approximation ratio of 2 instead. It prove $S_{i+k} - S_i \geq L$ for all i by contradiction by assuming if the starting time of job i+k $S_{i+k} - S_i < L$ instead, which by discrete time, it is $S_{i+k} - S_i \leq L - 1$, it then used to show $S_{i+k} \leq S_i + L - 1$. Then show the lower bound progress of job i+r at time T by using that upper bound of the starting time and to align the lower bound with the upeer bound of Peak(k+1, τ, s), which Peak(k+1, τ, s)>M by definition, which show that Mem(T)≥Peak(k+1, τ, s)>M, which cause the contradiction.

Then use $S_{i+k} - S_i \geq L$, we know that by induction from i=0, 1, 2, ..., $S_{nk}^{OPT} \geq nL$, for any i=qk+r, which q then is $\lfloor \frac{i}{k} \rfloor$, the i-th job startimng time $S_i^{OPT} = S_{qk+r}^{OPT} \geq S_{qk}^{OPT} \geq qL = \lfloor \frac{i}{k} \rfloor L$. Then compare the completion time of OPT and SPS, by using L=$2\lceil \frac{\tau}{2} \rceil \geq \lceil \tau \rceil$, to show the upper bound of $C_i^{SPS} \leq 2C_i^{OPT}$.

4.2.5 EC. 2.2 for Theorem 1

Assuming s=0 for EC 2.2.

Proposition EC. 1. Lower Bound of SIMS in asymptotic regime. Batch size B=$\lfloor M/(s+o) \rfloor$, and recall the SPS flowtime. Then $\frac{SIMS}{OPT} \geq \frac{SIMS}{SPS} = 2 - \epsilon$ when B, o tends to $+\infty$.

Proposition EC. 2. Lower Bound of SIMS approaching 3 when only o tends to $+\infty$. Define a time spacing d. Define M by expressing in d that M<2o. Then B<2, B then

=1 in SIMS. It then also show in staggering for 3 jobs in a batch in any other staggered schedule S also not OOM by entering at o-2d, o-d, o. Then $\frac{SIMS}{OPT} \geq \frac{SIMS}{S} \geq 3 - \epsilon$ when o tends to $+\infty$.

5 Conclusion and Acknowledgments

Back to Top

In summary, this paper makes a landmark contribution to the field of LLM serving optimization. It does provide a rigorous study on those floor functions and ceiling functions. It di make a clear style of using staggering to process the jobs instead of only mentioning about the batch processing, which could make a new direction on the variation of the Knapsack problem.

The comprehensive analysis detailed throughout this paper has significantly enriched my theoretical knowledge and reshaped my understanding of hardware-software co-design. I would like to express my sincere gratitude to you for providing this profound research framework and for taking the time to review this reflection.

6 References

Chen, Z., Ye, Y., & Zhou, Z. (2025). Adaptively Robust LLM Inference Optimization under Prediction Uncertainty. arXiv.Org. https://arxiv.org/abs/2508.14544

Adamaszek, A., Kociumaka, T., Marcin, P., & Pilipczuk, M. (2016). Hardness of approximation for strip packing. arXiv.Org. https://arxiv.org/abs/1610.07766

Wang, M., Ye, Y., & Zhou, Z. (2025). LLM Serving Optimization with Variable Prefill and Decode Lengths. arXiv.Org. https://arxiv.org/abs/2508.06133